

How To Use Modern MULTI-PROCESSOR APPLICATION PROGRAMS



V.P. Sampath works as a consultant who develops hardware/software co-design tools. He is an active senior member of IEEE and a member of Institution of Engineers

An application built with the hybrid model of parallel programming can run on a computer cluster using both OpenMP and message passing interface

Modern embedded applications are becoming complex and demanding with respect to code reuse, as platforms and applications are being developed and implemented rapidly. Implementation is a combination of hardware and software, and may change several times during a product's lifetime. Hence, efficient hardware alone is not enough. It needs the support of programming models and tools.

Modern field-programmable gate array (FPGA) devices are flexible, and can include up to tens of soft-core processors and IP blocks. The primary objective is a practical solution that is immediately usable in embedded product development.

An application program interface (API) is an abstraction that describes an interface for interaction between system components. In practice, it is a set of functions in a program, such as `send_data()` or `connect_channel()`. A common interface is needed to develop complex portable applications efficiently. The same functions must be available to programmers, regardless of API implementation, such as a PC running Linux and NIOS without an operating system (OS).

Consequently, most development and functional verification can be carried out on a workstation. And porting the tested software into target multi-processor system on chip (SoC) should go smoothly. However, some APIs make certain assumptions from the hardware platform, like utilisation of shared memory.

Overall functionality is specified for model-based techniques, and mapped to physical implementation consisting of various processors and fixed-function hardware IP blocks. In addition, efficient software platform services like OSes, drivers and compatibility layers are needed to implement an interface layer that provides suitable abstraction for both software and hardware blocks. These should also offer feasible performance, small memory and area requirements, fast code porting and re-mapping of application tasks.

Application tasks are implemented in processor programs and fixed-function hardware blocks using unified interface layer abstraction. Intel Atom, customisable TTA processors and Altera NIOS are a few examples.

Multi-core communications API (MCAPI) is targeted at inter-core communication, whereas multi-processor API (MPI) and sockets are developed for inter-computer communication. Thus, a principal design goal of MCAPI is to specify a low-latency API to enable efficient use of a network-on-chip. MCAPI's communication latencies and memory footprint are expected to be significantly lower than MPI's or sockets', at the expense of less flexibility.

MCAPIs also enable building of plug-and-play component software environment, development of portable, object-oriented, interoperable code, which is hardware-, OS-, network- and programming-language-independent.

For a system with multiple processor sub-systems, the following architecture decisions must be considered:

- Inter-processor communication
- Partitioning/sharing peripherals (locking required)
- Bandwidth and latency requirements
- May have inherited processor sub-system from another development team or third party
- Risk mitigation by reducing change, fulfilling latency and bandwidth requirements
- Real-time considerations
- If main processor is not real-time enabled, a real-time processor sub-system design partition/sandboxing can be added
- Break the system into smaller sub-systems to service tasks
- Smaller tasks to be designed easily
- Leverage software resources; sometimes a problem is resolved in less time by processor/software rather than hardware design, sequencers or state-machines
- Huge number of processor sub-systems can be implemented; bandwidth and latency can be tailored